

NASATurbofanLevelB2 (A level upgrade)

1. Introduction

The work addresses missing sensor data in the aircraft engine degradation dataset. The dataset contains 25 variables, including three operational settings and 21 sensor measurements, recorded over multiple operational cycles. Within a single operational setting (the first and the third were selected for the analysis – OS1 and OS3), the Operational Setting variables are considered constant and therefore not used in analysis. Measurements are performed on 100 individual units, each with a different number of time cycles. One of the sensors is assumed to have stopped emitting data (Sensor 17 was selected for the OS1 and Sensor 21 for the OS3), and the goal is to estimate its missing values using multivariate regression. The observations from individual sensors do not have a known physical representation. Key aspects include determining the most appropriate number of latent variables for regression and identifying the minimum number of required sensors.

The goal is to estimate the values of the missing sensor based on measurements from the remaining sensors, where each unit's measurements form a time series of varying length. Missing sensor values will be predicted via regression over a defined sliding time window, capturing the temporal context within each sequence. The sliding window (size will be the regression model hyperparameter) approach handles the problem of sequences with different lengths.

2. Materials and methods

2.1 Data description

Data come from the NASA Turbofan Engine Degradation Simulation Dataset (C-MAPSS). The inputs are structured as multivariate time series, tracking the complete run-to-failure degradation of multiple simulated turbofan jet engines. Each cycle provides 26 recorded values, including the engine identifier, three operational settings (control inputs), and measurements from 21 sensors (health indicators like temperature and pressure). The operating settings vary the number of operating conditions and fault modes.

2.2 Mathematical methods

Modelling goal of methods

The aim is to perform multivariate regression (and evaluate its quality), where the input is a window of measured sensors values (part of the time-series), and the outputs are values of the missing sensor of the respective window. Furthermore, the task is also to identify the minimal subset of sensors and the optimal number of latent variables required to predict the target sensor.

Mathematical methods used in modelling

The regression will be performed on windows. Each window of length w will include sensor data and contextual information – intercept and slope of each sensor linear approximation of the previous window (within the curve). The windows will act as samples. Principal Component Analysis (PCA) will be applied to extract latent variables that capture the main sources of variability among the flattened input samples. Principal Component Regression (PCR [1]) will be performed using the most significant

principal components to predict the target sensor values (based on the linear least squares). This approach not only reduces the dimensionality of the data but also mitigates multicollinearity among the sensor inputs. The key model hyperparameters are D (set of input sensors), window size w and k (number of PCs); which will be selected based on cross-validation performance. Mathematical formulation of setting PCR parameters is

$$\hat{\beta} = (\mathbf{P}_k^T \mathbf{X}^T \mathbf{X} \mathbf{P}_k)^{-1} \mathbf{P}_k^T \mathbf{X}^T \mathbf{Y},$$

where $\mathbf{P}_k \in \mathbb{R}^{[D] \cdot (w+2) \times k}$ is the orthogonal matrix of the first k principal components (each column is a principal component vector, which are eigenvectors of the covariance matrix of $\mathbf{X}$), $\mathbf{X} \in \mathbb{R}^{n \times [D] \cdot (w+2)}$ is the matrix of input features and $\mathbf{Y} \in \mathbb{R}^{n \times w}$ is the target matrix. Prediction for the new measurement $\mathbf{x}_{\text{new}}$ is given as $\mathbf{x}_{\text{new}}^T \mathbf{P}_k \hat{\beta}$.

In addition to PCR, Partial Least Squares, PLS [2], regression will be applied as an alternative latent-variable method. Unlike PCR, which maximizes the variance explained in the input data, PLS directly extracts components that maximize the covariance between predictors and target variables. This makes PLS potentially more efficient in cases where the most informative latent directions are not aligned with the directions of maximal input variance. The mathematical formulation of PLS regression is as follows. The underlying model is

$$\mathbf{X} = \mathbf{T} \mathbf{P}^T + \mathbf{E}, \quad \mathbf{Y} = \mathbf{U} \mathbf{Q}^T + \mathbf{F},$$

where $\mathbf{X} \in \mathbb{R}^{n \times [D] \cdot (w+2)}$ is the matrix of input features, $\mathbf{Y} \in \mathbb{R}^{n \times w}$ is the target matrix, $\mathbf{T} \in \mathbb{R}^{n \times k}$ and $\mathbf{U} \in \mathbb{R}^{n \times k}$ are the scores matrices (latent variables extracted from $\mathbf{X}$ and $\mathbf{Y}$, $\mathbf{P} \in \mathbb{R}^{[D] \cdot (w+2) \times k}$ and $\mathbf{Q} \in \mathbb{R}^{w \times k}$ are the loading matrices and $\mathbf{E}$ and $\mathbf{F}$ are the residual matrices. The latent components are obtained iteratively by maximizing the covariance between each pair of score vectors (t_i, u_i) :

$$t_i = \mathbf{X} \boldsymbol{\beta}_i, \quad u_i = \mathbf{Y} \boldsymbol{\alpha}_i, \quad \max \text{cov}(t_i, u_i),$$

where the $\boldsymbol{\beta}_i \in \mathbb{R}^{[D] \cdot (w+2)}$ is the weight vector for $\mathbf{X}$ and $\boldsymbol{\alpha}_i \in \mathbb{R}^w$ is the weights vector for $\mathbf{Y}$. The weights matrix is obtained as $\mathbf{B} = [\boldsymbol{\beta}_1, \dots, \boldsymbol{\beta}_k] \in \mathbb{R}^{[D] \cdot (w+2) \times k}$ and the regression coefficient matrix is given as $\hat{\mathbf{B}} = \mathbf{B} (\mathbf{P}^T \mathbf{B})^{-1} \mathbf{Q}^T$. The prediction for a new measurement $\mathbf{x}_{\text{new}}$ is given as $\mathbf{x}_{\text{new}}^T \hat{\mathbf{B}}$.

In addition to PCR and PLS, the regression will be performed by the kernel PCR (kPCR) method. Kernel Principal Component Regression is a nonlinear extension of Principal Component Regression that maps the original input data into a high-dimensional feature space. The distance between two samples is defined by the kernel function $k(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$. Such functions are, for example, Gaussian

RBF $k(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$, or polynomial kernel $k(x_i, x_j) = (x_i^T x_j + c)^d$. Principal

component analysis is then performed in this transformed space to extract nonlinear principal components that capture complex structures in the data $K_c v_k = n \lambda_k v_k$. The resulting components are then used in a regression model to predict the dependent variable $t_k = K_c v_k$. The approach enables to model nonlinear relationships between predictors and responses, usually yielding improved predictive accuracy and robustness compared to linear PCR.

Model calibration, validation and testing strategies

Model calibration, validation, and testing strategy will be performed using a combination of fixed splits and three-fold cross-validation. The input to the procedure will be clean data – zscore standardized window samples with addition of contextual information. The dataset will be randomly divided (while preserving particular curves – samples coming from the same curve will be in the same part) into five

parts (1–5), with part 4 reserved as a fixed validation set (validation group) and part 5 as a fixed test set (testing group). Three-fold cross-validation is applied to parts 1–3 (training-calibration group) as follows: for each fold, two parts are used to train the PCR model for multiple combinations of hyperparameters (k , w and D), and one part is used as a calibration set to select the best-performing fold. After cross-validation, the fixed validation set (part 4) is used to select, among the best-performing folds with different parameters setups, the best-performing parameter combination (PCR parameters setup, k , w and D), and the final model is evaluated on the fixed test set (part 5) to provide an objective assessment of performance. R^2 will be computed for training data group, the Q^2 will be computed for validation and testing data groups. The Root mean squared error (RMSE) will be computed for all (training, validation and testing) groups. While R^2 measures how well the model fits the training data, Q^2 evaluates its predictive performance on independent validation or test data using the calibration (training) mean, making it a more reliable indicator of generalization and helping to detect overfitting. The selection of best-performing folds will be based on the Q^2 value. Figure 1 shows the strategy.

Next, after determining the optimal window size w and the set of sensors D , the effect of the number of latent variables (k) on R^2 and RMSE will be explored, to identify the dependence of the model complexity on its performance. Furthermore, the autocorrelation of the regression prediction residuals will be analysed, to verify that the residuals behave as white noise and that no systematic temporal patterns remain unmodeled. A comparison of selected time series (predictions versus ground truth) will also be conducted, to visualise the model's ability to reproduce the trend of the missing sensor values based on the observed data.

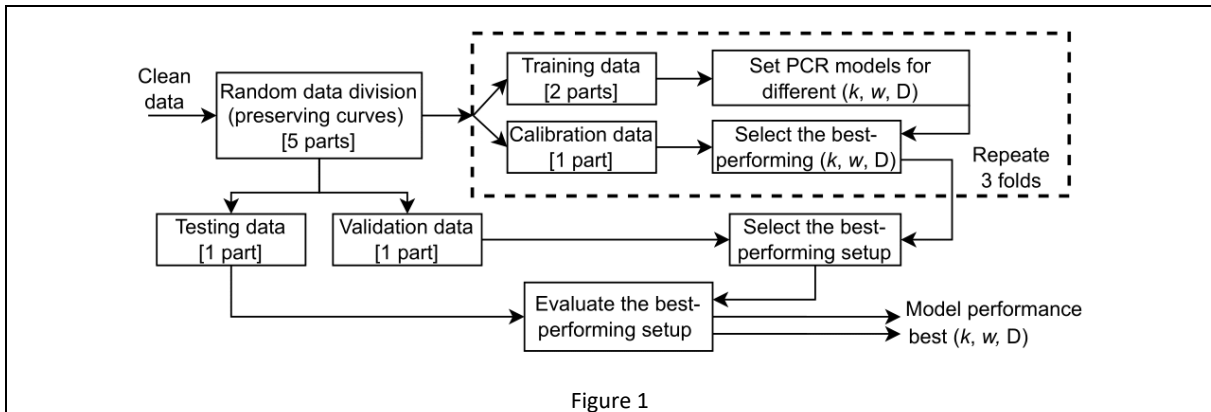


Figure 1

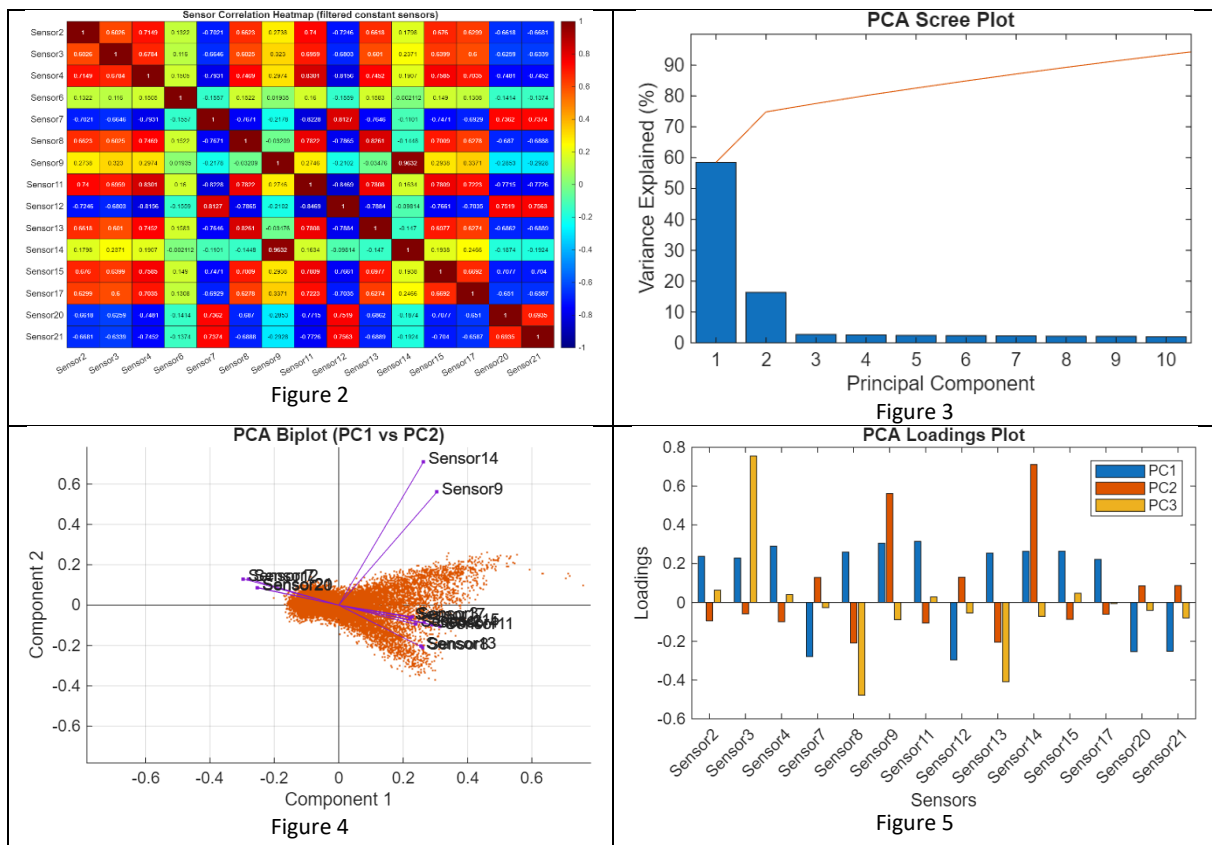
3. Results and discussion

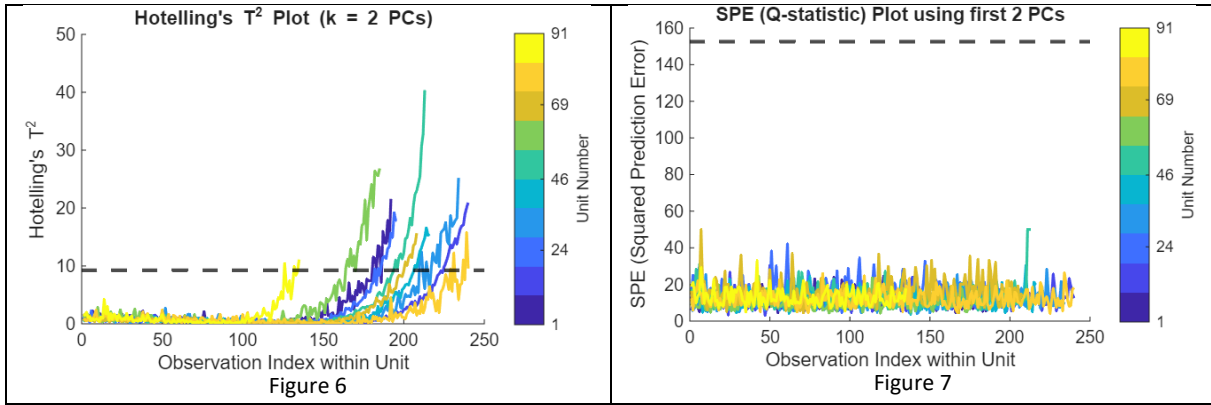
3.1 Exploratory data analysis

For initial PCA analysis, all measurements were consolidated into a single matrix (for OS1: 20,631 observations by 21 sensor features). The distributions of sensors observations of OS1 showed us that that sensors 1, 5, 6, 10, 16, 18, 19 are constant (for OS3 the Sensors 1, 5, 10, 16, 18, 19); next, the measurements are differently scaled and located, so the standardization is needed. Figure 2 shows a correlation heatmap (without constant sensors) for sensors of OS1. It can be seen that there are mutually correlated sensors (e.g. 2 and 3 and 4). Furthermore, there are two highly correlated sensors 9 and 14. The target sensor 17 is correlated with sensors 2, 3, 4, 8, 11, 13, 15. Figure 3 shows principal components explained variance (OS1). The PCA was computed after z-standardization (each unit and each sensor separately) and filtering constant sensors. It is visible that over 80% of information is given

by the first three components. Figure 4 shows PCA biplot with 2 PCs (OS1). Sensors 3 and 17 (target) are closely aligned, indicating that they convey similar information along the directions of maximum variance. It can be expected that sensor 3 will be crucial for the regression. Sensors 2, 4, 11 and 15 have high importance in the 1st PC and low in the 2nd. Sensors 7, 12, 20 and 21 show opposite loadings on the 1st PC. This suggests that the 1st PC captures contrasting behaviour between the two sensor groups across the samples. These findings are also visible in Figure 5 (OS1), which presents the loading sizes according to PCs. The T^2 control chart, Figure 6, shows the temporal evolution of Hotelling's T^2 for individual units (OS1). Ten representative unit trajectories were selected. In the second half of the trajectories, T^2 values increase gradually and systematically, eventually surpassing the critical threshold, determined according to Hadian & Rahimifard [3]. Figure 7 shows the Squared Prediction Error (SPE) for the OS1. Despite some minor peaks, all measurements remain below the SPE critical limit (Jackson & Mudholkar [4]).

Due to the distributions of measured sensor values according to histograms, it is necessary to standardize the data (zscore standardization). It can also be assumed that not all sensors are necessary for the prediction of the missing sensors, since there are constant sensors. The regression should be performed in a window-based approach.





3.2 Data pre-treatment

As a first step of data pre-treatment, the constant sensors are removed (for OS1 the Sensors 1, 5, 6, 10, 16, 18 and 19 are removed, for OS3 the Sensors 1, 5, 10, 16, 18, 19). Next, the dataset is randomly shuffled at the unit level and split (also at the unit level) into the three groups. The first group is the training-calibration set (60 % of the units within the dataset operational setting), the second group is the validation set (20 % of the units within the dataset operational setting) and the third group is the testing set (20 % of the units within the dataset operational setting).

As a further step a zscore standardization is needed. The mean and standard deviation are computed from the training-calibration set, and these values are used to normalize all three groups. This ensures that the validation and testing data are standardized consistently with the training-calibration data. Standardization removes the effect of different sensor scales so that all variables contribute equally to subsequent analyses (e.g. PCA). Figure 8 shows histograms of all data (time) points after zscore standardization.

Missing values in the dataset are not explicitly addressed, because the dataset does not contain any such records (samples with unknown values of one or more sensors).

Since the sensor time-series of individual units have different lengths, regression will be performed by using a sliding-window approach. For this reason, each time-series will be split into overlapping windows. To preserve contextual information, each window (and thus each variable except the target variable) is augmented with the slope and intercept of a linear approximation of the preceding window (within the Unit). For the first window of each unit, zero context information is used. In this way, each window (except the first one) is not processed as an isolated sequence of points but as a sequence with information about the previous linear trend estimate. Figure 9 illustrates an example using Sensor 3 data for the OS1. The coloured points represent sensor values within windows, and the dashed line shows the linear context derived from the preceding window.

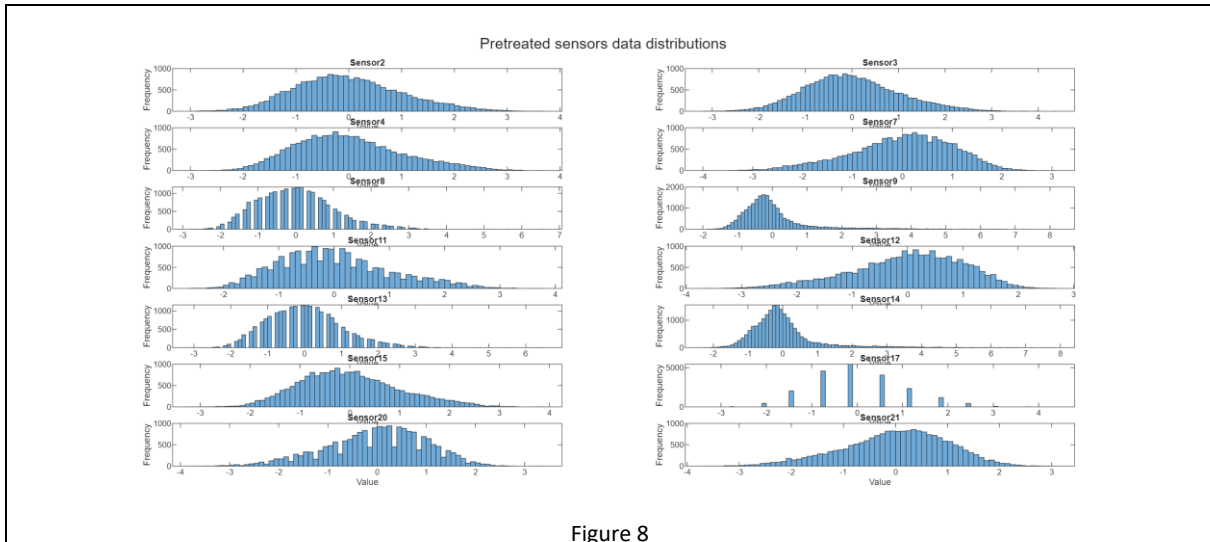


Figure 8

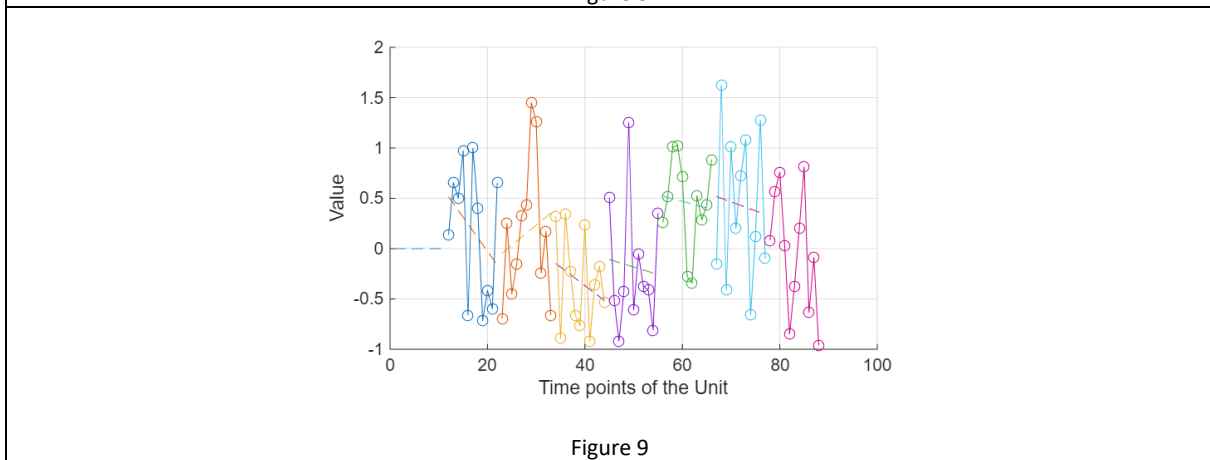


Figure 9

3.3 Modelling results

Initial experimental evaluation procedure

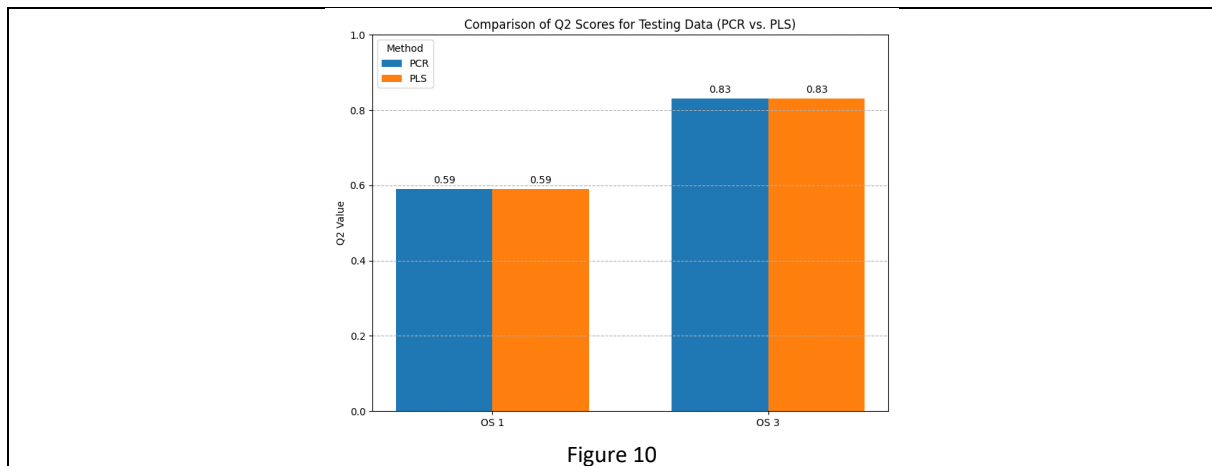
The aim of the initial experimental evaluation was primarily to estimate the approximate number of latent variables required for the PCR and PLS models and to assess whether it is possible to identify the trend of the missing sensor's measurements based on a limited subset of sensors providing input data to the model.

Model calibration, validation, and testing were conducted using a combination of fixed data splits and three-fold cross-validation. The dataset was divided into five parts, with parts 1–3 used for training and calibration, part 4 for validation, and part 5 for final testing. The input data were zscore standardized and to the input windows was added information about context. During the cross validation, the models parameters were set based on the training data and the best-performing fold was selected based on the calibration data. The optimal hyperparameters (k – number of latent variables, w – window size, D – set of sensors) were selected based on the validation Q^2 , and the final model performance was evaluated on the test set using Q^2 , and RMSE.

The initial experimental evaluation was performed by setting the hyperparameters window size (w) to 5 and 10, and the number of latent variables (k) to 3 and 5. The evaluation was carried out using all

non-constant sensors (excluding the target sensor) as well as a subset of two sensors that, according to the biplot analysis, were expected to carry the most information in common with the target sensor. Tables A1, A2, A3, and A4 present the results of this initial experimental evaluation for operational settings (OS) 1 and 3, where in operational setting 1 the missing target sensor was S17, and in operational setting 3 the target (missing) sensor was S21. For both operational settings, PCR and PLS models were experimentally evaluated. The best performance was achieved for a window size of 5 and five latent variables when all sensors were included. This finding can thus be used to guide further analysis of the appropriate number of latent variables, indicating that the optimal number of latent variables and window size is likely below 10.

Figure 10 visualise the Q^2 on testing data.



It is also important to note that although regression based on only two input sensors (operational setting 1 – sensors 3 and 11; operational setting 3 – sensors 7 and 20) produced slightly worse results (in terms of RMSE and Q^2 on validation data), the performance degradation was not substantial, suggesting that even two sensors are sufficient to identify the trend of the missing sensor.

Analysis of latent number of variables

Tables 1 show Q^2 values on validation data depending on the number of latent variables. These results were obtained for both OS 1 and OS 3 with a window size of 5 on the best-performing folds identified during the initial experimental evaluations. The results show that the PLS model required fewer latent variables to achieve better performance compared to PCR, especially in OS 1. Table 2 shows that PLS exhibits lower cumulative explained variance in the training data than PCR, as it prioritizes components that best predict the target sensor values rather than those explaining the largest variance in the input sensor values. Both PCR and PLS models exhibit a gradual increase in explained variance in training data as the number of latent variables increases. For OS 1, the variance grows only slightly beyond three components, matching the plateau observed in Q^2 values. For OS 3, the variance continues to increase more noticeably, in line with the improvement of Q^2 . Overall, both models show stable behaviour after several latent variables, confirming that the main structure of the data is captured within the first few components. These results suggest that both models effectively represent the underlying data patterns with a limited number of latent variables, with PLS showing slightly higher efficiency. The consistent trends between explained variance and Q^2 further indicate good model regularization and support the selection of a small number of latent variables for optimal complexity and generalization. The kernel PCR with quadratic polynomial kernel shows the worst prediction performance. Changing kernel to RBF significantly has worsened the quality. For that reason using

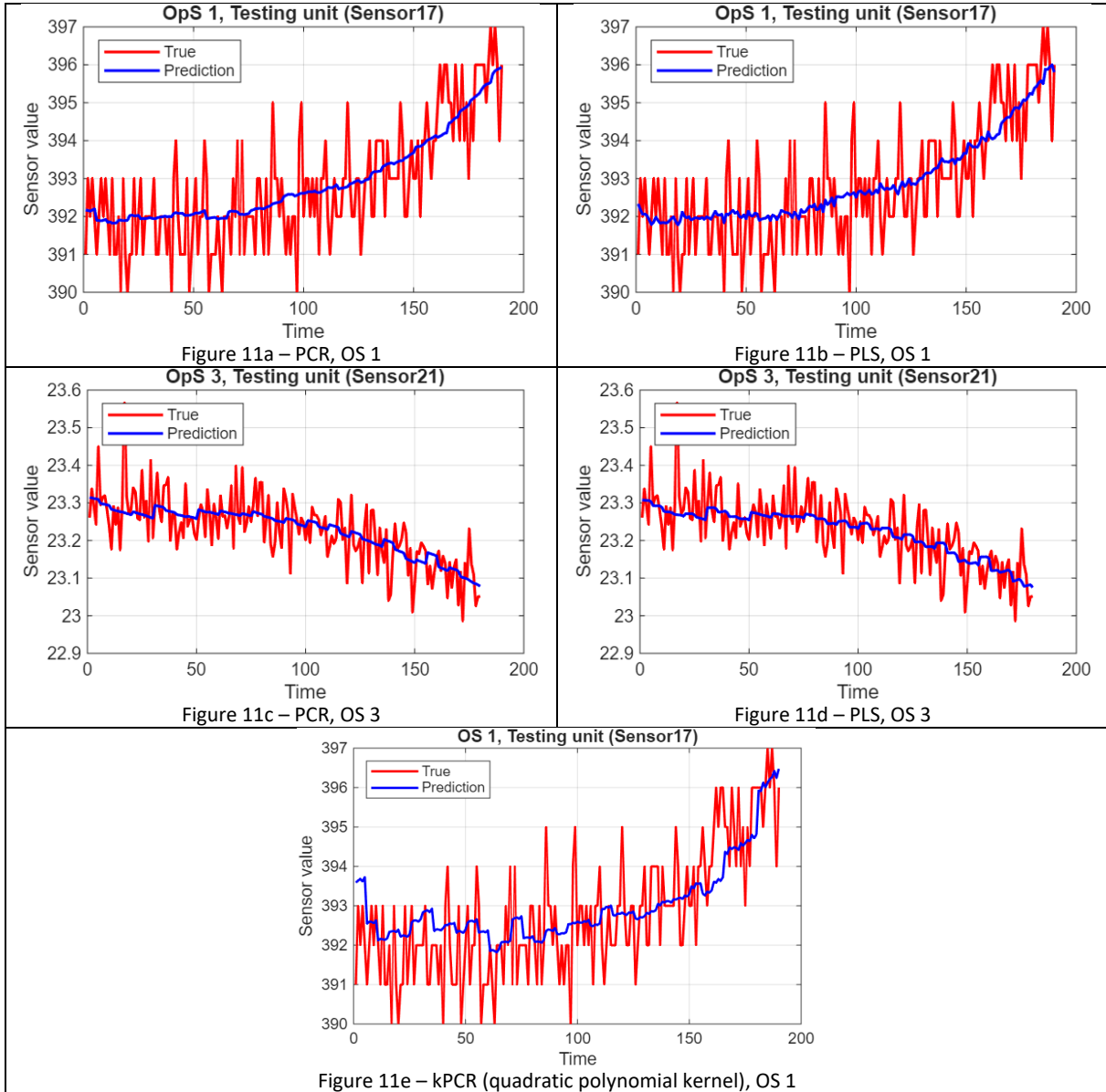
kernel methods is not an appropriate approach for this task. The optimal number of latent variables was determined based on the criterion of stabilization of predictive performance. Specifically, the number was defined as the point beyond which further increases in the number of latent variables resulted in a marginal change (less than 0.001) in the predictive performance (Q^2) of the model on the validation dataset.

Table 1 – Q^2 values on validation data (dependency on number of latent variables)									
Latent variables	1	2	3	4	5	6	7	8	9
OS1, PCR, target 17, w 5, fold 3, all sensors	0.5769	0.6041	0.6041	0.6042	0.6043	0.6043	0.6042	0.6041	0.6040
OS1, PLS, target 17, w 5, fold 3, all sensors	0.5879	0.6043	0.6043	0.6043	0.6021	0.6014	0.6008	0.6007	0.6004
OS1, kPCR, target 17, w5, fold 1, all sensors	0.1431	0.1563	0.1707	0.4974	0.5040	0.5584	0.5589	0.5601	0.5610
OS3, PCR, target 21, w 5, fold 1, all sensors	0.1685	0.7706	0.8103	0.8246	0.8328	0.8330	0.8332	0.8333	0.8333
OS3, PLS, target 21, w 5, fold 1, all sensors	0.7374	0.7996	0.8297	0.8316	0.8322	0.8328	0.8343	0.8351	0.8350

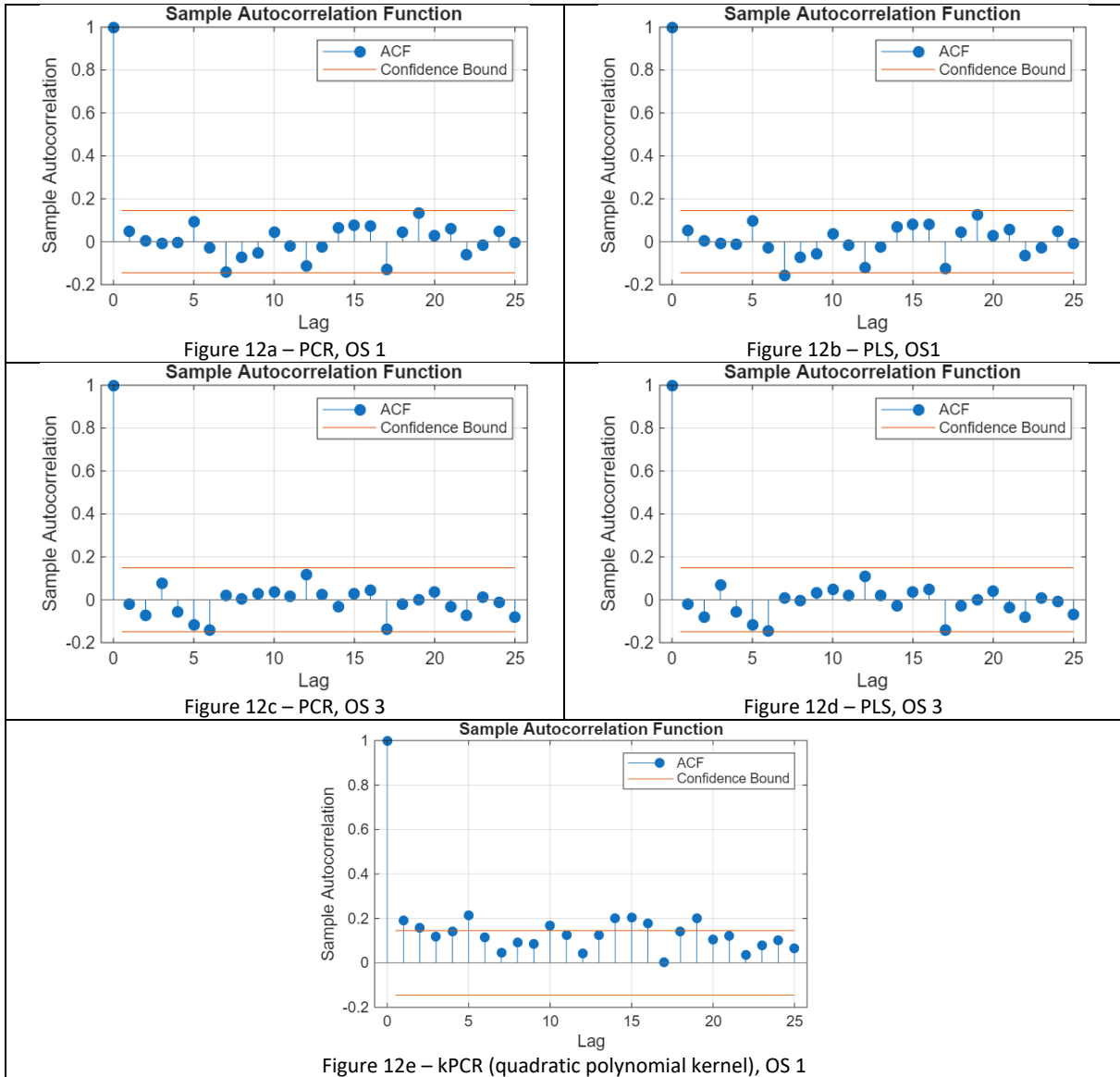
Table 2 – cumulative explained variance in training data (dependency on number of latent variables) [%]									
Latent variables	1	2	3	4	5	6	7	8	9
OS1, PCR, target 17, w 5, fold 3, all sensors	63.45	76.71	77.74	78.66	79.43	80.16	80.84	81.43	82.01
OS1, PLS, target 17, w 5, fold 3, all sensors	63.39	76.71	77.48	78.18	78.58	79.03	79.45	79.99	80.62
OS3, PCR, target 21, w 5, fold 1, all sensors	49.96	73.19	78.64	83.41	85.04	85.87	86.65	87.36	88.00
OS3, PLS, target 21, w 5, fold 1, all sensors	35.10	73.08	78.25	80.36	84.60	85.52	85.98	87.07	87.07

Results visualisation and residuals analysis

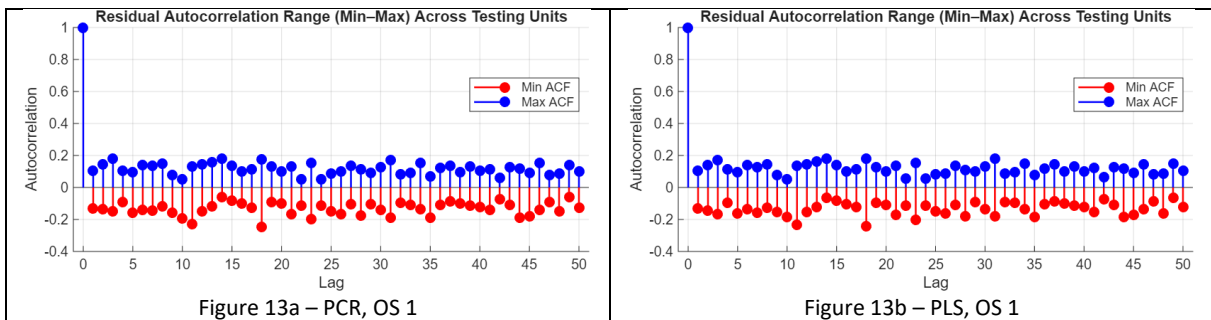
As a next step, there was performed an analysis on residuals for the best-performing configurations (given by the previous section) of models. Figures 11a, 11b, 11c, 11d and 11e show example comparisons of predictions and true signal data. It can be seen that the trend of the missing sensors is well captured, however the high-frequency component is not captured. The question is: is it a high-frequency noise or a periodical component? Probably noise, we will test the hypotheses below.



To answer such a question about the origin of residuals, an autocorrelation analysis was performed. Figures 12a, 12b, 12c, 12d and 12e show autocorrelation plots for corresponding samples predictions and true sensor data from previous Figures. It is visible that for lags greater than zero, the autocorrelation function is bounded between -0.2 and 0.2.



Figures 13a, 13b, 13c, 13d and 13e show the minimum and maximum values of each lag of autocorrelation functions computed over all testing Units. The values of lags (greater or equal one) are bounded between -0.2 and 0.2, and their average values are approximately equal zero. From this can be concluded that the residuals behave like white noise and the models are capturing the information of the target (missing) sensor.



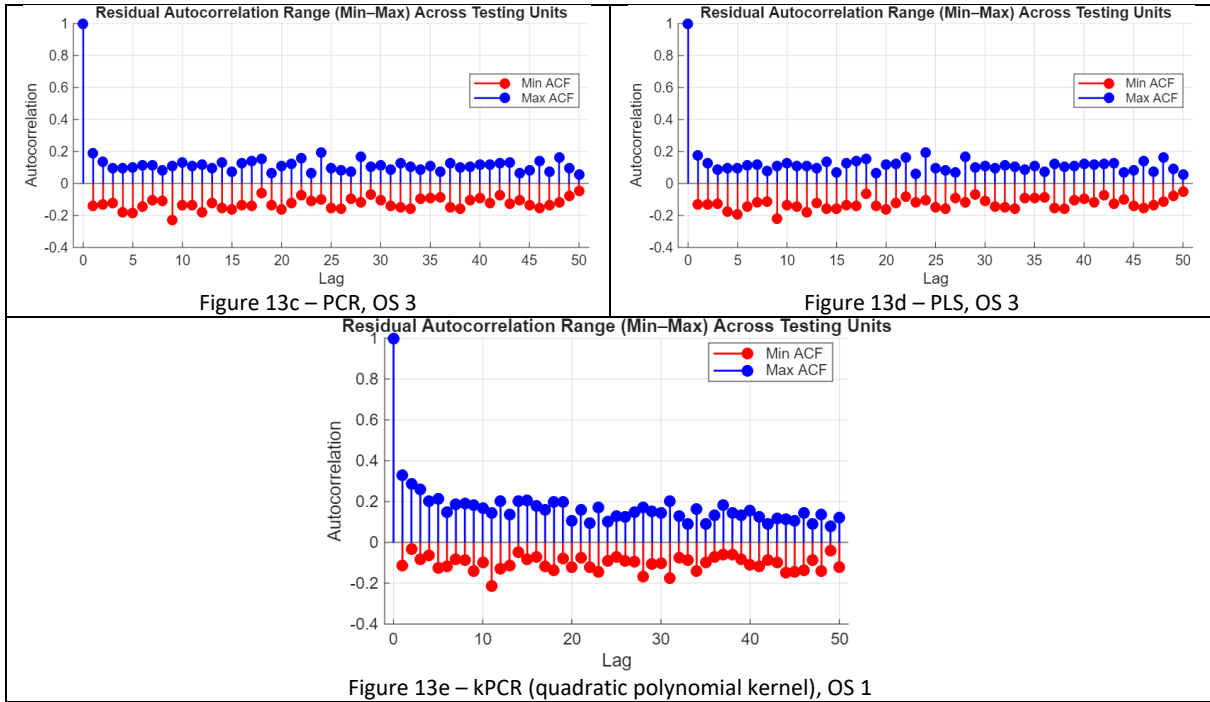
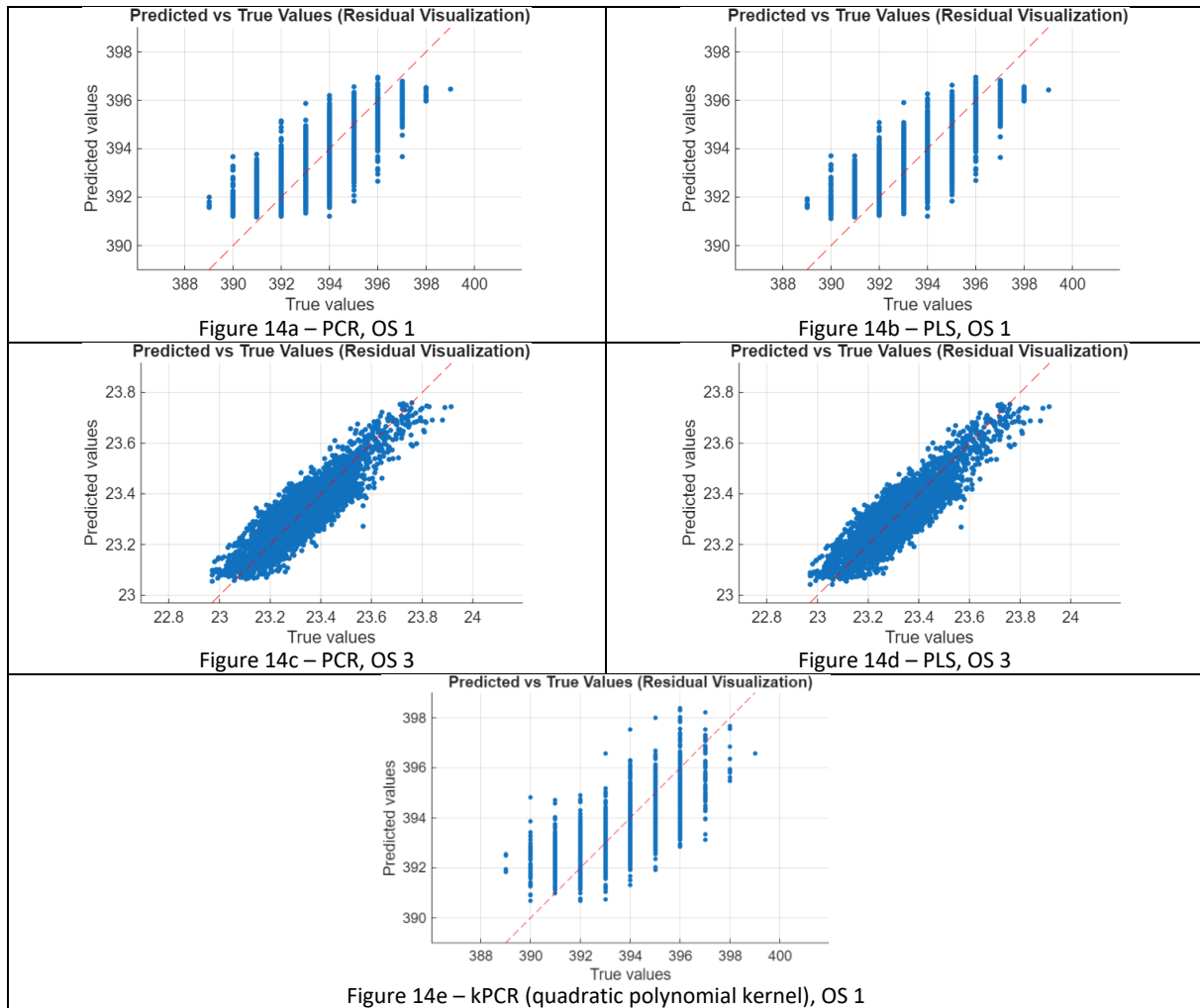


Table 3 summarizes the results of statistical tests on residuals for the PCR and PLS models across OS 1 and 3. The Ljung–Box tests [5] assessed autocorrelation in the residuals at several lag values and showed that the majority of tested units did not reject the null hypothesis, indicating that the residuals are largely uncorrelated. The number of failures was low, typically 1–2 out of 20 units, with minimal differences between models and operating states. The ARCH tests [6], which check for constant variance in the residuals, showed zero or very few failures, indicating that the residual variance is stable and heteroskedasticity is not a significant concern. Overall, the residuals of both models behave approximately as white noise, supporting the correct specification of the models. The results are consistent across PCR and PLS models and across different OS, suggesting robustness. Individual minor failures can be considered random deviations and do not represent a systematic issue. The quadratic polynomial kernel PCR resulted into autocorrelated residuals, which was confirmed by statistical hypotheses testing.

Table 3 – number of testing Units rejecting H_0 of statistical tests on residuals ($\alpha = 0.05$)					
Model, OS	Ljung-Box (5 Lags)	Ljung-Box (10 Lags)	Ljung-Box (20 Lags)	Ljung-Box (20 Lags)	ARCH test
PCR, 1	1/20	1/20	2/20	2/20	0/20
PLS, 1	1/20	1/20	2/20	2/20	0/20
kPCR, 1	7/20	8/20	9/20	8/20	7/20
PCR, 3	2/20	2/20	1/20	0/20	1/20
PLS, 3	2/20	2/20	1/20	0/20	1/20

Figures 14a, 14b, 14c, 14d and 14e show scatter plots of the dependency of predictions on true values. It can be visible that for the OS 1 the target sensor S17 does not produce continuous values, however it changes its measurements in steps. This could be the reason for the worse performance than the prediction of S21 in OS 3.



From Tables 1 and 3 and Figures 11 to 14, it is visible that the kernel method is not a good choice for the task. The best regression performance was achieved by the PLS method; furthermore, the PLS required fewer latent variables compared to the PCR.

Conclusion

Modelling showed that both PCR and PLS were able to accurately reconstruct the missing trend in sensor data, even with a limited number of (two) input sensors on both operational settings. The best performance was achieved when using a window size of five samples and five latent variables. Crucially, the PLS model outperformed PCR and required fewer components. Both models captured the main structure of the data already in the first latent variables, which was confirmed by Q^2 on validation data and explained variance. The models were thus able to generalize. Analysis of residuals and autocorrelation then showed that the residuals behaved like white noise, which means that the models captured key information about the missing data without significant autocorrelation or heteroskedasticity. The adequacy of both models was also confirmed by statistical tests (Ljung–Box and ARCH). The kernel PCR models, which produced autocorrelated residuals and showed poor prediction accuracy, completely failed, confirming (including statistical hypothesis testing) that they are unsuitable for this task. Overall, the PLS model offered the best balance between complexity and performance, and thus proved to be the most robust and efficient method for reconstructing missing sensor measurements under various operating conditions.

Codes and data

https://github.com/drvojtex/ADAML_NASATurbofanLevelB2

Authors contributions statement

Vojtěch Drahý

- Implemented all source codes; excluding utils/R2eval.m, utils/autocorrEval.m and utils/resAnalysis.m.
- Wrote the report; excluding the description of kPCR in Mathematical methods subsection.

Jarkko Komulainen

- Implemented functions utils/R2eval.m, utils/autocorrEval.m and utils/resAnalysis.m.
- Wrote the description of kPCR in Mathematical methods subsection.

References

- [1] <https://doi.org/10.1002/0471725331>
- [2] https://doi.org/10.1007/978-1-62703-059-5_23
- [3] <https://doi.org/10.1016/j.cie.2019.03.021>
- [4] <https://doi.org/10.1080/00401706.1979.10489779>
- [5] 10.1093/biomet/65.2.297
- [6] 10.2307/1912773

The Grammarly tool was used while writing the report. [<https://app.grammarly.com/>]

Table A1 – Operational setting 1 (target Sensor 17), PCR					
Hyperparameters setup	Fold	Training (RMSE, R2)	Calibration (RMSE, Q2)	Validation (RMSE, Q2)	Testing (RMSE, Q2)
window size = 5 k = 3 all sensors	1	0.9478, 0.6099	0.9470, 0.6105		
	2	0.9459, 0.6160	0.9508, 0.5996		
	3	0.9486, 0.6037	0.9454, 0.6232	0.9716, 0.6041	
window size = 5 k = 5 all sensors	1	0.9472, 0.6105	0.9472, 0.6104		
	2	0.9455, 0.6163	0.9505, 0.5999		
	3	0.9483, 0.6040	0.9449, 0.6236	0.9713, 0.6043	0.9643, 0.5899
window size = 10 k = 3 all sensors	1	0.9463, 0.5973	0.9461, 0.5957		
	2	0.9451, 0.6018	0.9484, 0.5881		
	3	0.9467, 0.5905	0.9456, 0.6092	0.9715, 0.5865	
window size = 10 k = 5 all sensors	1	0.9462, 0.5974	0.9462, 0.5956		
	2	0.9450, 0.6019	0.9483, 0.5881		
	3	0.9465, 0.5907	0.9457, 0.6091	0.9715, 0.5866	
window size = 5 k = 3 S 3, 11	1	0.9692, 0.5922	0.9670, 0.5940		
	2	0.9666, 0.5990	0.9719, 0.5817		
	3	0.9689, 0.5866	0.9674, 0.6055	1.001, 0.5800	
window size = 5 k = 5 S 3, 11	1	0.9690, 0.5924	0.9672, 0.5938		
	2	0.9666, 0.5990	0.9720, 0.5816		
	3	0.9688, 0.5867	0.9674, 0.6056	1.001, 0.5799	
window size = 10 k = 3 S 3, 11	1	0.9608, 0.5849	0.9568, 0.5865		
	2	0.9584, 0.5906	0.9616, 0.5766		
	3	0.9586, 0.5802	0.9614, 0.5962	0.9901, 0.5709	
window size = 10 k = 5 S 3, 11	1	0.9608, 0.5850	0.9568, 0.5866		
	2	0.9583, 0.5907	0.9619, 0.5763		
	3	0.9586, 0.5803	0.9615, 0.5961	0.9902, 0.5707	

Table A2 – Operational setting 1 (target Sensor 17), PLS					
Hyperparameters setup	Fold	Training (RMSE, R2)	Calibration (RMSE, Q2)	Validation (RMSE, Q2)	Testing (RMSE, Q2)
window size = 5 k = 3 all sensors	1	0.9470, 0.6106	0.0471, 0.6105		
	2	0.9482, 0.6141	0.9538, 0.5971		
	3	0.9478, 0.6044	0.9451, 0.6234	0.9714, 0.6043	0.9641, 0.5901
window size = 5 k = 5 all sensors	1	0.9453, 0.6120	0.9480, 0.6097		
	2	0.9462, 0.6157	0.9553, 0.5958		
	3	0.9460, 0.6059	0.9464, 0.6224	0.9740, 0.6021	
window size = 10 k = 3 all sensors	1	0.9462, 0.5974	0.9466, 0.5953		
	2	0.9503, 0.5973	0.9548, 0.5824		
	3	0.9462, 0.5910	0.9465, 0.6085	0.9725, 0.5857	
window size = 10 k = 5 all sensors	1	0.9444, 0.5989	0.9482, 0.5939		
	2	0.9484, 0.5989	0.9567, 0.5807		
	3	0.9445, 0.5924	0.9477, 0.6075	0.9744, 0.5841	
window size = 5 k = 3 S 3, 11	1	0.9673, 0.5938	0.9634, 0.5970		
	2	0.9668, 0.5988	0.9720, 0.5816		
	3	0.9656, 0.5894	0.9670, 0.6059	0.9988, 0.5817	
window size = 5 k = 5 S 3, 11	1	0.9670, 0.5941	0.9636, 0.5968		
	2	0.9666, 0.5990	0.9724, 0.5813		
	3	0.9654, 0.5896	0.9669, 0.6059	0.9991, 0.5815	
window size = 10 k = 3 S 3, 11	1	0.9608, 0.5850	0.9576, 0.5859		
	2	0.9624, 0.5872	0.9652, 0.5734		
	3	0.9580, 0.5808	0.9614, 0.5962	0.9896, 0.5713	
window size = 10 k = 5 S 3, 11	1	0.9603, 0.9582	0.9582, 0.5853		
	2	0.9619, 0.5876	0.9655, 0.5731		
	3	0.9578, 0.5810	0.9614, 0.5963	0.9900, 0.5709	

Table A3 – Operational setting 3 (target Sensor 21), PCR					
Hyperparameters setup	Fold	Training (RMSE, R2)	Calibration (RMSE, Q2)	Validation (RMSE, Q2)	Testing (RMSE, Q2)
window size = 5 k = 3 all sensors	1	0.0632, 0.8065	0.0631, 0.8266	0.0659, 0.8103	
	2	0.0631, 0.8186	0.0612, 0.8139		
	3	0.0622, 9.8197	0.0645, 0.8040		
window size = 5 k = 5 all sensors	1	0.0612, 0.8181	0.0612, 0.8328	0.0619, 0.8328	0.0608, 0.8332
	2	0.0615, 0.8279	0.0606, 0.8176		
	3	0.0608, 0.8274	0.0619, 0.8199		
window size = 10 k = 3 all sensors	1	0.0625, 0.8020	0.0622, 0.8238	0.0641, 0.8143	
	2	0.0626, 0.8141	0.0608, 0.8079		
	3	0.0612, 0.8175	0.0634, 0.8033		
window size = 10 k = 5 all sensors	1	0.0610, 0.8114	0.0609, 0.8313	0.0616, 0.8283	
	2	0.0612, 0.8219	0.0604, 0.8103		
	3	0.0606, 0.8208	0.0617, 0.8277		
window size = 5 k = 3 S 7, 20	1	0.0654, 0.7919	0.0657, 0.8122	0.0662, 0.8084	
	2	0.0659, 0.8025	0.0648, 0.7914		
	3	0.0652, 0.8021	0.0662, 0.7935		
window size = 5 k = 5 S 7, 20	1	0.0652, 0.7936	0.0657, 0.8130	0.0660, 0.8098	
	2	0.0657, 0.8034	0.0645, 0.7935		
	3	0.0650, 0.8037	0.0661, 0.7944		
window size = 10 k = 3 S 7, 20	1	0.0634, 0.7964	0.0634, 0.8174	0.0640, 0.8146	
	2	0.0637, 0.8071	0.0626, 0.7961		
	3	0.0630, 0.8068	0.0642, 0.7982		
window size = 10 k = 5 S 7, 20	1	0.0634, 0.7970	0.0634, 0.8176	0.0640, 0.8148	
	2	0.0637, 0.8074	0.0626, 0.7996		
	3	0.0630, 0.8073	0.0642, 0.7984		

Table A4 – Operational setting 3 (target Sensor 21), PLS					
Hyperparameters setup	Fold	Training (RMSE, R2)	Calibration (RMSE, Q2)	Validation (RMSE, Q2)	Testing (RMSE, Q2)
window size = 5 k = 3 all sensors	1	0.0618, 0.8134	0.0617, 0.8343	0.0624, 0.8297	
	2	0.0619, 0.8254	0.0607, 0.8170		
	3	0.0613, 0.8246	0.0629, 0.8139		
window size = 5 k = 5 all sensors	1	0.0712, 0.8177	0.0613, 0.8363	0.0620, 0.8322	0.0609, 0.8328
	2	0.0614, 0.8286	0.0605, 0.8183		
	3	0.0610, 0.8268	0.0620, 0.8192		
window size = 10 k = 3 all sensors	1	0.0617, 0.8071	0.0615, 0.8279	0.0623, 0.8246	
	2	0.0617, 0.8191	0.0605, 0.8096		
	3	0.0611, 0.8180	0.0627, 0.8077		
window size = 10 k = 5 all sensors	1	0.0612, 0.8100	0.0611, 0.8301	0.0619, 0.8269	
	2	0.0612, 0.8221	0.0604, 0.8104		
	3	0.0608, 0.8196	0.0619, 0.8127		
window size = 5 k = 3 S 7, 20	1	0.0656, 0.7908	0.0658, 0.8118	0.0666, 0.8077	
	2	0.0659, 0.8025	0.0648, 0.7915		
	3	0.0656, 0.7993	0.0666, 0.7914		
window size = 5 k = 5 S 7, 20	1	0.0653, 0.7930	0.0655, 0.8134	0.0660, 0.8096	
	2	0.0655, 0.8044	0.0644, 0.7940		
	3	0.0652, 0.8016	0.0663, 0.7932		
window size = 10 k = 3 S 7, 20	1	0.0636, 0.7948	0.0635, 0.8165	0.0642, 0.8136	
	2	0.0638, 0.8069	0.0627, 0.7958		
	3	0.0634, 0.8039	0.0646, 0.7959		
window size = 10 k = 5 S 7, 20	1	0.0635, 0.7953	0.0635, 0.8166	0.06415, 0.8138	
	2	0.0636, 0.8077	0.0626, 0.7966		
	3	0.0633, 0.8046	0.0645, 0.7963		